

RAPPORT TECHNIQUE DE PROJET

Vision industrielle reliée à un API pour le contrôle d'étiquettes

CODESYS | OPC UA | Python | OpenCV

Réalisé par	Nouredine Qarafi
École	ENSAM Meknès
Filière	Génie électromécanique
Projet	Contrôle d'étiquettes de bouteilles
Année universitaire	2026 / 2027

Résumé et sommaire

00 Résumé du projet

J’ai réalisé un prototype de laboratoire qui relie un API simulé sous CODESYS à un client Python de traitement d’images. Un événement de détection déclenche le contrôle d’une image de bouteille ; un classificateur fondé sur les contours recherche l’étiquette, puis envoie une demande d’éjection à l’API si la forme attendue est absente. Le programme API et une IHM affichent l’état du système.

00.1 Objectif

L’objectif est de démontrer toute la chaîne de signaux, depuis la détection de proximité jusqu’à l’indication « acceptée / rejetée ». CODESYS Control Win V3 simule l’automate, deux images locales remplacent les prises de vue et OPC UA transporte les variables. Le rapport décrit la séquence réalisée et les observations visibles dans les captures fournies.

00.2 Périmètre de la démonstration

Limite du prototype. Les images proviennent de fichiers locaux. La configuration présentée ne comporte ni caméra réelle, ni convoyeur, ni capteur de bouteille, ni éjecteur physique.

Sommaire

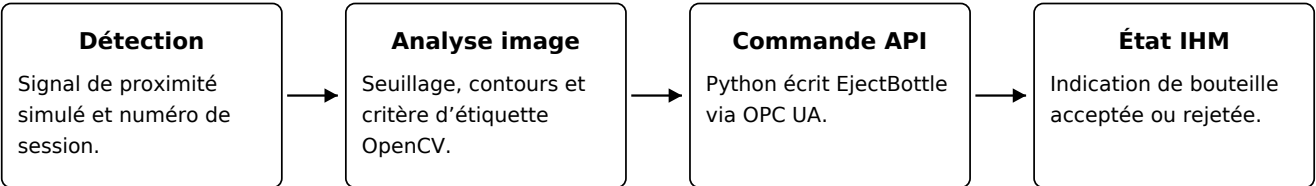
01	Architecture et communication	3
02	Séquence API et commande d’éjection	4
03	Sessions et indications d’état	5
04	Traitement d’image et décision Python	6
05	IHM et essais observés	7
06	Limites et conclusion	8

Architecture et communication

01 Rôles des systèmes et variables échangées

CODESYS Control Win V3 x64 héberge l’API simulé et publie les variables de l’application sur son serveur OPC UA. Le processus Python est le client. UaExpert permet de parcourir séparément les nœuds exposés et de lire leurs valeurs courantes.

1.1 Chaîne des signaux



1.2 Variables échangées

Bottledetected et sessinNumber. L’API transmet à Python l’événement de détection et un numéro de session.

EjectBottle. Python écrit la demande de rejet ; l’API l’efface après la temporisation d’éjection.

exit_script. L’API publie une condition d’arrêt que la boucle du client lit.

accepted et unaccepted. Des expressions booléennes de l’API commandent les indications IHM.

1.3 Liaison OPC UA

L’API simulé sous CODESYS est le serveur OPC UA et Python est le client. Le point d’accès par défaut est **opc.tcp://DESKTOP-QUBUSE2:4840**. UaExpert affichait séparément les variables de l’application avec l’état Good lors de la capture.

#	SERVER	NODE ID	NODE PATH	DISPLAY NAME	VALUE	D
1	OPCUAServer@DE...	NS4 String var C...	Programs/...edetector	Bottledetected	true	Boolea
2	OPCUAServer@DE...	NS4 String var C...	Programs/...ectBottle	EjectBottle	false	Boolea
3	OPCUAServer@DE...	NS4 String var C...	Programs/.../Proximity	Proximity	false	Boolea
4	OPCUAServer@DE...	NS4 String var C...	Programs/...ect_relay	eject_relay	true	Boolea
5	OPCUAServer@DE...	NS4 String var C...	Programs/P...xit_script	exit_script	false	Boolea
6	OPCUAServer@DE...	NS2 Numeric 6095	DeviceTop...ineAccess	OnlineAccess		Null

Figure 1. Extrait monochrome agrandi de la vue UaExpert montrant les variables API publiées et leurs valeurs.

Séquence API et commande d'éjection

02 Événement bouteille et temporisations

Le programme Ladder produit une impulsion de proximité simulée, mémorise l'événement sur front montant et remet le bit de détection à zéro après une durée définie. En cas de rejet, Python écrit EjectBottle ; cette variable commande le relais logique d'éjection jusqu'à la remise à zéro par temporisation.

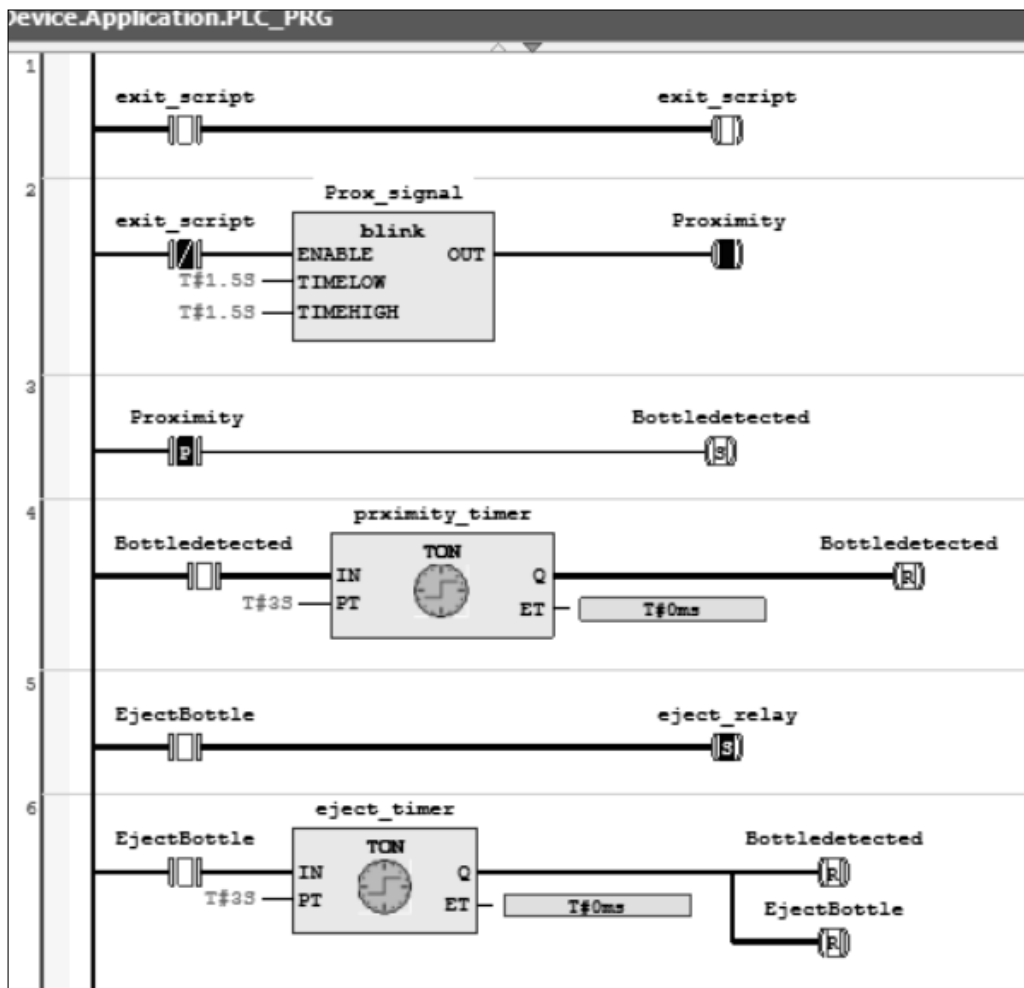


Figure 2. Réseaux Ladder principaux : simulation de proximité, mémorisation, temporisation de détection de 3 s, relais d'éjection et remise à zéro après 3 s (capture fournie).

2.1 Détection

Un bloc Blink simule Proximity. Son front montant active Bottledetected, que proximity_timer remet à zéro après une consigne nominale T#3s.

2.2 Éjection

Quand Python écrit EjectBottle, l'API active eject_relay. eject_timer efface ensuite les commandes de détection et d'éjection après une consigne nominale T#3s.

2.3 Périmètre

Le capteur et le relais sont des signaux logiciels. Leur synchronisation avec un convoyeur réel n'a pas été validée.

Gestion des sessions et indications d'état

03 Un traitement par session détectée

Un compteur incrémente sa valeur lorsque Bottledetected devient actif, se réinitialise au seuil cinq, puis transfère sa valeur courante vers la variable nommée **sessinNumber** dans le programme. Python lit cette variable avec Bottledetected et la compare à sa valeur locale last_session avant de traiter une nouvelle image.

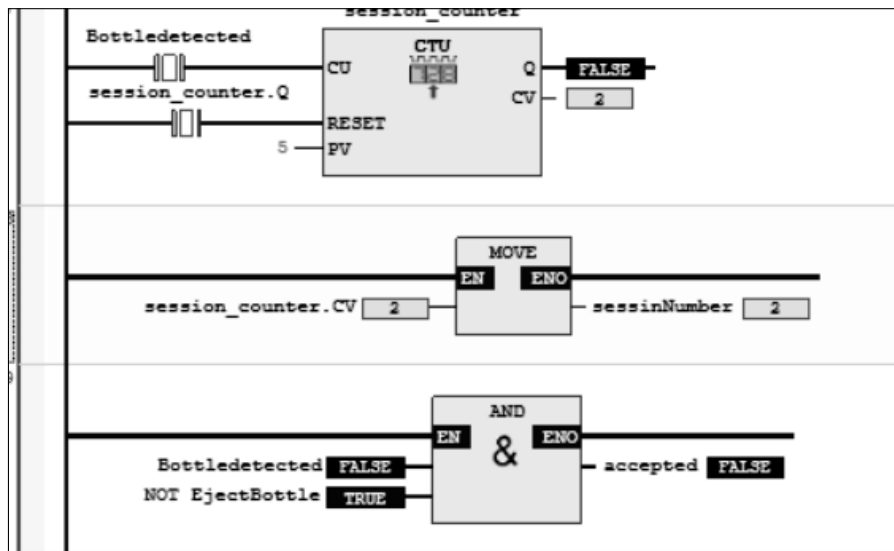


Figure 3. Compteur de sessions CODESYS, transfert MOVE vers sessinNumber et premiers réseaux d'indication (capture fournie).

3.1 Synchronisation avec le client

La boucle Python vérifie simultanément Bottledetected et le changement de numéro de session avant de choisir l'image suivante. La variable porte l'orthographe sessinNumber dans le code et dans la vue API.

3.2 Indications acceptée et rejetée



Figure 4. Logique affichée en ligne : accepted = Bottledetected AND NOT EjectBottle ; unaccepted = Bottledetected AND EjectBottle. La condition de rejet est active sur la capture.

Ces indicateurs décrivent l'état instantané de l'API. Ils ne conservent pas le résultat qualité après la remise à zéro de la détection. Un historique ou un compteur de pièces demanderait des variables de résultat distinctes.

Méthode de traitement d'image Python

04 Chaîne de traitement

Le client Python sélectionne l'une de deux images locales, la charge en niveaux de gris avec OpenCV et applique un seuillage adaptatif par moyenne. L'extraction des contours et l'approximation polygonale forment ensuite un critère géométrique simple d'étiquette. Le même fichier est également chargé en couleur pour dessiner un contour reconnu.

4.1 Acquisition

Un choix aléatoire entre deux fichiers JPEG locaux simule l'acquisition. Le fichier choisi est chargé en niveaux de gris, puis en couleur pour tracer un contour.

4.2 Seuillage

Le seuillage adaptatif par moyenne d'OpenCV convertit l'image en binaire avec un voisinage de 11 pixels et $C = 5$.

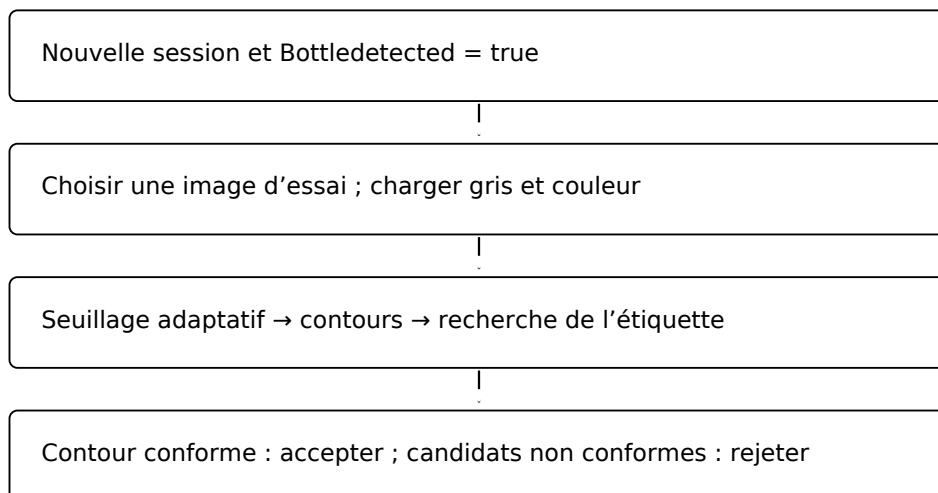
4.3 Recherche des contours

findContours utilise RETR_TREE et CHAIN_APPROX_NONE. Chaque contour est mesuré par son aire et son périmètre, puis approximé par approxPolyDP avec $\epsilon = 0,02 \times \text{périmètre}$.

4.4 Critère d'étiquette

Un candidat comporte quatre sommets et une aire strictement comprise entre 142 et 31320 pixels dans le code actuel.

4.5 Logigramme de décision



Le classificateur demande l'éjection lorsqu'aucun contour ne satisfait le critère d'étiquette. En cas de rejet, le client écrit la variable booléenne EjectBottle via OPC UA et incrémente un compteur local de rejets.

Cas limite du code actuel : si findContours ne renvoie aucun contour, check_label conserve should_eject = False et retourne False. Une image sans aucun contour est donc acceptée par la version actuelle ; il faut corriger cette valeur par défaut et tester ce cas.

IHM et essais observés

05 Vue opérateur

L'IHM CODESYS réunit la commande d'arrêt, la proximité simulée et l'éjection avec des bouteilles sur fond rouge et vert. Elle permet de suivre les variables API pendant que le processus Python réalise la classification.

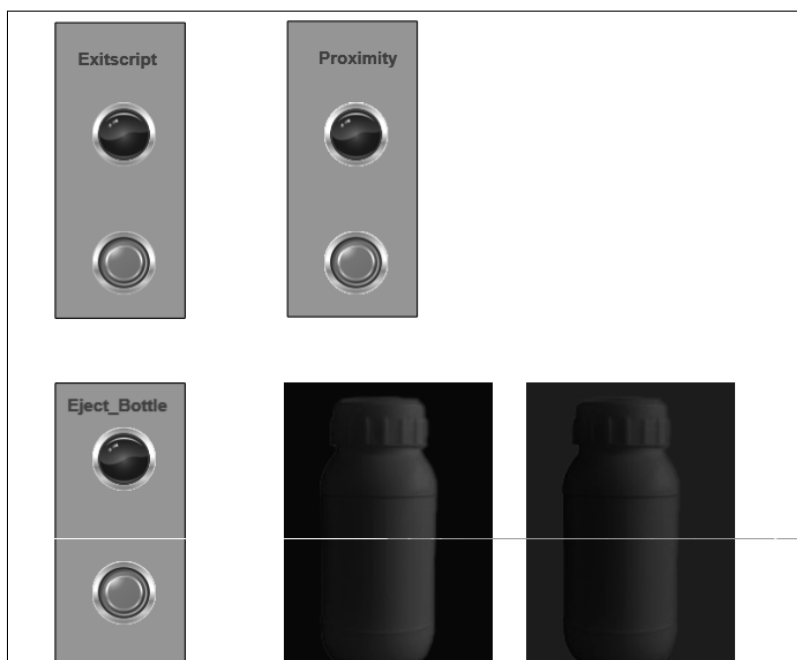


Figure 5. Extrait monochrome de l'IHM. Le panneau de gauche représente le rejet et celui de droite l'acceptation.

5.1 Indices tirés de la séance enregistrée

```
-----
EjectBottle activated
3
-----
no label detected . activate Ejection
-----
EjectBottle activated
4
█
```

Figure 6. Extrait de la console Python de la même capture : « no label detected », activation de EjectBottle et compteur de rejets atteignant quatre.

Communication. UaExpert montre des variables API lisibles avec l'état Good durant la séance capturée.

Cas rejeté. La console Python affiche l'absence d'étiquette et l'activation de EjectBottle ; la capture API affiche unaccepted = true.

Cas accepté. L'expression d'acceptation et l'indication IHM existent ; aucun taux de réussite mesuré sur un lot indépendant n'a été fourni.

Limites et conclusion

06 Limites techniques et essais à prévoir

01. Source des images : le choix aléatoire entre deux fichiers remplace une prise de vue déclenchée ; aucun réglage de caméra, éclairage ou position sur convoyeur n'est présent.

02. Classification : les seuils d'aire et de quatre sommets sont réglés pour les exemples. Les taux de faux acceptés et faux rejets, ainsi que la tenue à la rotation, aux reflets ou aux masquages, ne sont pas mesurés.

03. Synchronisation : Python incrémente sa propre valeur last_session au lieu de reprendre exactement sessinNumber. Un décalage après réinitialisation pourrait ignorer ou répéter une bouteille.

04. Reprise : la boucle client ne gère ni reconnexion, ni échec de lecture d'image, ni politique d'expiration, ni attente ; ces points sont nécessaires pour un fonctionnement prolongé.

6.1 Diagnostic d'exécution

```
2026-09-29T10:35:48.497Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:48.497Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:48.826Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:48.826Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:49.262Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:49.262Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:50.563Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:50.563Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:51.110Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:51.547Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:51.547Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:53.178Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:53.178Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:53.834Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:53.834Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
```

Figure 7. Une capture ultérieure de CODESYS Control Win V3 montre plusieurs messages « Scheduler Tick / [FAILED] » (capture fournie).

Ce journal n'annule pas l'échange OPC UA et la séquence de rejet observés, mais ne permet pas de conclure à une exploitation continue stable. Un nouvel essai devrait relier les journaux API, les numéros de session, les décisions Python et les résultats IHM sur un lot défini.

6.2 Conclusion

J'ai intégré un API simulé sous CODESYS, un client OPC UA, un classificateur Python/OpenCV et une IHM dans une démonstration du rejet de bouteilles sans étiquette détectée. Les captures fournies montrent des variables API accessibles et l'exécution d'une commande d'éjection. Ce prototype permet d'étudier l'interface entre vision industrielle et automatisme ; la suite consiste à corriger les cas limites et à valider l'ensemble sur un jeu d'images contrôlé avec une exécution fiable.

6.3 Référence pédagogique

Mouhammad Hamsho et Kemalaldin Hamso, PLC Basic Machine Vision | From Scratch, Udemy (page consultée le 29 septembre 2026) : <https://www.udemy.com/course/plc-basic-machine-vision-from-scratch/>

Éléments du projet : plc_connecting_code.py et captures CODESYS, UaExpert, IHM et console fournis avec ce rapport. Les figures proviennent des captures du projet.